

PROJECT DOCUMENTATION

AXI4 Memory SystemVerilog Verification Project

Design verification workflow, bug fixes, and testbench architecture

PREPARED BY

Youssef Emad Ali

ID: 479

AXI4 | SystemVerilog | Verification

Requirements Testing Matrix								Test Cases	
Req ID	Requirement Description	Type	Priority	TC ID	Expected Output	Actual Output	Status	TC ID	TC Description
REQ-001	Test single beat write	Functional	High	TC-001	resp = 00 , data written	resp = 00	Verified	TC-001	AWLEN = 0 and write data
REQ-002	Test single beat read	Functional	High	TC-002	resp = 00 , data read	resp = 00	Verified	TC-002	ARLEN = 0 and read data
REQ-003	Test multiple beats write	Functional	High	TC-003	resp = 00 , data beats written	resp = 00	Verified	TC-003	AWLEN > 0 and write data
REQ-004	Test mutiple beats read	Functional	High	TC-004	resp = 00, data beats read	resp = 00	Verified	TC-004	ARLEN > 0 and read data
REQ-005	Test out of boundary address	Functional	Critical	TC-005	resp = 01	resp = 01	Verified	TC-005	apply address out of boundary and check response
Summary									
Total Requi	5								
Verified	5								
Failed	0								
Pending	0								
Coverage %	100.0%								

Functional Coverage Plan

Functional Coverage Plan								
CovGrp ID	Covergroup Name	Coverpoint Name	Bin / Cross Definition	Type	Legal Values / Range	Weight	Target Hits	Status
CG-001	cg	c_AWADDR	cover all write address in ranges	bin	16'd0 : 16'd4091	1	1	Hit
CG-001	cg	c_ARADDR	cover all read address in ranges	bin	16'd0 : 16'd4091	1	1	Hit
CG-002	cg	c_AWLEN	cover all write beat length in ranges	bin	8'd0 : 8'd255	1	1	Hit
CG-002	cg	c_ARLEN	cover all read beat length in ranges	bin	8'd0 : 8'd255	1	1	Hit
CG-003	cg	c_WDATA	cover all written data in ranges	bin	2^32	1	1	Hit
CG-003	cg	c_BRESp	cover the response of write operation	bin	1'b0	1	1	Hit
CG-003	cg	c_RRESP	cover the response of read operation	bin	1'b0	1	1	Hit
CG-003	cg	c_AWSIZE	AWSIZE = 2	bin	1'd2	1	1	Hit
CG-003	cg	c_ARSIZE	ARSIZE = 2	bin	1'd2	1	1	Hit
CG-003	cg	cross_write_add_len	cross between the write addresses and length	cross	-	1	1	Hit
CG-003	cg	cross_read_add_len	cross between the read addresses and length	cross	-	1	1	Hit

Contents

The document follows the verification plan, coverage plan, verification flow, and SystemVerilog testbench components.

P1	Verification Plan	02
P2	Functional Coverage Plan	03
01	Verification & Bug Fixes	05
02	Interface	08
03	Transaction	09
04	Generator	11
05	Driver	12
06	Scoreboard	15
07	Monitor	21
08	Environment	23
09	Testbench	24
10	Logs & Waveforms	26
11	Coverage	30

Step I walked through to verify the AXI4: the verification started from the memory module, then moved to single-beat transfers, and finally to multiple-beat bursts.

1

Verify the memory module and found two bugs

```
// Memory write
always @(posedge clk) begin
    if (!rst_n) // reset is negedge and this was flipped (BUG FIX)
        mem_rdata <= 0;
    else if (mem_en) begin
        if (mem_we)
            memory[mem_addr] <= mem_wdata;
        else
            mem_rdata <= memory[mem_addr]; // it was reading the previous address (BUG FIX)
    end
end
```

BUG FIX 1 Corrected the active-low reset condition and fixed the memory read path.

Result

The memory block now resets using the intended active-low polarity and returns data from the selected memory word instead of using an incorrect read path.

2

Verified single beat write and read then found latency bug

```

R_ADDR: begin
    // Start first read
    if (read_addr_valid && !read_boundary_cross) begin
        mem_en <= 1'b1;
        mem_addr <= (read_addr >> 2); // Convert to word address
    end
    read_state <= R_WAIT;
end
R_WAIT : begin // wait for memory to read the correct address (BUG FIX)
    read_state <= R_DATA;
end
R_DATA: begin
    // Present read data
    if (read_addr_valid && !read_boundary_cross) begin
        RDATA <= mem_rdata;
        RRESP <= 2'b00; // OKAY
    end else begin
        RDATA <= {DATA_WIDTH{1'b0}};
        RRESP <= 2'b10; // SLVERR
    end
end

RVALID <= 1'b1;
RLAST <= (read_burst_cnt == 0);

if (RREADY && RVALID) begin
    RVALID <= 1'b0;

    if (read_burst_cnt > 0) begin
        // Continue burst
        read_addr <= read_addr + read_addr_incr;
        read_burst_cnt <= read_burst_cnt - 1'b1;
    end

    // Start next read
    if (read_addr_valid && !read_boundary_cross) begin
        mem_en <= 1'b1;
        mem_addr <= ((read_addr + read_addr_incr) >> 2);
    end
    read_state <= R_WAIT; // wait to capture the right address (BUG FIX)
end

```

BUG FIX 2 A wait state is inserted between the memory request and the AXI read-data response.

```

reg [2:0] read_state;
localparam R_IDLE = 3'd0,
            R_ADDR = 3'd1,
            R_WAIT = 3'd2, // added wait state to fix the timing between the slave and the memory (BUG FIX)
            R_DATA = 3'd3;

```

BUG FIX 3 R_WAIT was added to the read-state encoding to model the required memory latency.

3

Verified multiple beats and found the last bug

```
// Address boundary check (4KB boundary = 12 bits)
assign write_boundary_cross = ((write_addr & 12'hFFF) + (write_burst_cnt << write_size)) > 12'hFFF;
assign read_boundary_cross = ((read_addr & 12'hFFF) + (read_burst_cnt << read_size)) > 12'hFFF;
// used the cnt instead of len because in the logic it gets decremented every beat (BUG FIX)
```

BUG FIX 4 The 4 KB boundary check uses the active burst counter because the counter is decremented on every beat.

Verification progression

**Memory**

Reset + read path

**Single beat**

Read latency

**Multiple beats**

Boundary logic

The interface groups the AXI4 channels into one reusable SystemVerilog interface and exposes dedicated modports for the testbench master, DUT slave, and monitor.

```
interface axi4_if (input bit CLK);
    logic ARESETn;
    logic [15:0] AWADDR;
    logic [7:0] AWLEN;
    logic [2:0] AWSIZE;
    logic AWVALID;
    logic AWREADY;
    logic [31:0] WDATA;
    logic WLAST;
    logic WVALID;
    logic WREADY;
    logic [1:0] BRESP;
    logic BVALID;
    logic BREADY;
    logic [15:0] ARADDR;
    logic [7:0] ARLEN;
    logic [2:0] ARSIZE;
    logic ARVALID;
    logic ARREADY;
    logic [31:0] RDATA;
    logic [1:0] RRESP;
    logic RLAST;
    logic RVALID;
    logic RREADY;
endinterface
```

INTERFACE 1 Signal declarations

```
modport master (input CLK,
    input ARESETn,
    output AWADDR,
    output AWLEN,
    output AWSIZE,
    output AWVALID,
    input AWREADY,
    output WDATA,
    output WLAST,
    output WVALID,
    input WREADY,
    input BRESP,
    input BVALID,
    output BREADY,
    output ARADDR,
    output ARLEN,
    output ARSIZE,
    output ARVALID,
    input ARREADY,
    input RDATA,
    input RRESP,
    input RLAST,
    input RVALID,
    output RREADY);
```

INTERFACE 2 Master modport

```
modport slave (input CLK,
    input ARESETn,
    input AWADDR,
    input AWLEN,
    input AWSIZE,
    input AWVALID,
    output AWREADY,
    input WDATA,
    input WLAST,
    input WVALID,
    output WREADY,
    output BRESP,
    output BVALID,
    input BREADY,
    input ARADDR,
    input ARLEN,
    input ARSIZE,
    input ARVALID,
    output ARREADY,
    output RDATA,
    output RRESP,
    output RLAST,
    output RVALID,
    input RREADY);
```

INTERFACE 3 Slave modport

```
modport monitor (input CLK,
    input ARESETn,
    input AWADDR,
    input AWLEN,
    input AWSIZE,
    input AWVALID,
    input AWREADY,
    input WDATA,
    input WLAST,
    input WVALID,
    input WREADY,
    input BRESP,
    input BVALID,
    input BREADY,
    input ARADDR,
    input ARLEN,
    input ARSIZE,
    input ARVALID,
    input ARREADY,
    input RDATA,
    input RRESP,
    input RLAST,
    input RVALID,
    input RREADY);
endinterface
```

INTERFACE 4 Monitor modport

The transaction class models AXI read/write stimulus, burst attributes, payload data, legal constraints, and functional coverage used by the generator and scoreboard.

```
package signals_pkg;

✓ typedef enum bit {
    WRITE_OP,
    READ_OP
} operation_e;

✓ class signals;

    operation_e      operation;
    rand logic [15:0] AWADDR;
    rand logic [7:0]  AWLEN;
    rand logic [2:0]  AWSIZE;
    ✓ rand logic [31:0] WDATA[];
        logic        WLAST;
        logic [1:0]   BRESP;
    rand logic [15:0] ARADDR;
    rand logic [7:0]  ARLEN;
    ✓ rand logic [2:0] ARSIZE;
        logic        RDATA[];
        logic [1:0]   RRESP;
        logic         RLAST;

    ✓ constraint address_c {
        AWADDR inside {[16'h0000:16'h0FFC]};
        ARADDR inside {[16'h0000:16'h0FFC]};
        AWADDR[1:0] == 2'b00;
        ARADDR[1:0] == 2'b00;
    }
    ✓ constraint size_c {
        AWSIZE == 3'd2;
        ARSIZE == 3'd2;
    }
    constraint wdata_size_c {
        WDATA.size() == AWLEN + 1;
    }
endclass
```

TRANSACTION 1 Fields and randomized transaction state

```
    c_BRESP : coverpoint BRESP {
        bins b1 = {2'b00};
        bins b2 = {2'b10};
    }
    c_RRESP : coverpoint RRESP {
        bins b1 = {2'b00};
        bins b2 = {2'b10};
    }
    c_AWSIZE : coverpoint AWSIZE {
        bins b1 = {3'd2};
    }
    c_ARSIZE : coverpoint ARSIZE {
        bins b1 = {3'd2};
    }
    croos_wire_add_len : cross c_AWADDR, c_AWLEN;
    croos_read_add_len : cross c_ARADDR, c_ARLEN;

endgroup

function new;
    cg = new();
endfunction
endclass

endpackage
```

TRANSACTION 3 Response/size coverage and covergroup

This portion defines legal address/size behavior and data-array constraints so generated write and read bursts remain consistent with the AXI memory configuration.

```
covergroup cg with function sample (logic [31:0] wdata = 32'b0);
  c_AWADDR : coverpoint AWADDR {
    bins b1 = {[16'd0 : 16'd511]};
    bins b2 = {[16'd512 : 16'd1023]};
    bins b3 = {[16'd1024 : 16'd1535]};
    bins b4 = {[16'd1536 : 16'd2047]};
    bins b5 = {[16'd2048 : 16'd2559]};
    bins b6 = {[16'd2560 : 16'd3071]};
    bins b7 = {[16'd3072 : 16'd3583]};
    bins b8 = {[16'd3584 : 16'd4092]};
  }
  c_ARADDR : coverpoint ARADDR {
    bins b1 = {[16'd0 : 16'd511]};
    bins b2 = {[16'd512 : 16'd1023]};
    bins b3 = {[16'd1024 : 16'd1535]};
    bins b4 = {[16'd1536 : 16'd2047]};
    bins b5 = {[16'd2048 : 16'd2559]};
    bins b6 = {[16'd2560 : 16'd3071]};
    bins b7 = {[16'd3072 : 16'd3583]};
    bins b8 = {[16'd3584 : 16'd4092]};
  }
  c_AWLEN : coverpoint AWLEN {
    bins b0 = {8'd0};
    bins b1 = {[8'd1 : 8'd63]};
    bins b2 = {[8'd64 : 8'd127]};
    bins b3 = {[8'd128 : 8'd191]};
    bins b4 = {[8'd192 : 8'd255]};
  }
  c_ARLEN : coverpoint ARLEN {
    bins b0 = {8'd0};
    bins b1 = {[8'd1 : 8'd63]};
    bins b2 = {[8'd64 : 8'd127]};
    bins b3 = {[8'd128 : 8'd191]};
    bins b4 = {[8'd192 : 8'd255]};
  }
  c_WDATA : coverpoint wdata {
    bins b1 = {[32'h00000000 : 32'h1FFFFFFF]};
    bins b2 = {[32'h20000000 : 32'h3FFFFFFF]};
    bins b3 = {[32'h40000000 : 32'h5FFFFFFF]};
    bins b4 = {[32'h60000000 : 32'h7FFFFFFF]};
    bins b5 = {[32'h80000000 : 32'h9FFFFFFF]};
    bins b6 = {[32'hA0000000 : 32'hBFFFFFFF]};
    bins b7 = {[32'hC0000000 : 32'hDFFFFFFF]};
    bins b8 = {[32'hE0000000 : 32'hFFFFFFF]};
  }
}
```

TRANSACTION 2

Randomization constraints for aligned addresses, transfer size, burst length, and data arrays.

The generator creates transaction objects, randomizes them, and sends successful transactions to the driver through a mailbox for execution.

```

package gen_pkg;
import signals_pkg::*;

class generator;

    mailbox #(signals) gen2drv;
    mailbox #(int) drv2gen;

    int num_sig = 3000;
    bit done = 0;

    task gen();
        signals sig;
        int token;
        repeat (num_sig) begin
            sig = new();
            if (!sig.randomize()) begin
                $fatal(1, "Transaction randomization failed");
            end
            $display("[GEN] write_addr=%h write_data=%p read_addr=%h",
                sig.AWADDR, sig.WDATA, sig.ARADDR);

            gen2drv.put(sig);
            drv2gen.get(token);

        end
        done = 1;
    endtask
endclass
endpackage

```

GENERATOR Transaction randomization loop and mailbox handoff to the driver.

The driver controls the AXI master-side pins. Initialization drives deterministic default values and waits for reset release before traffic begins.

```
package driver;
import signals_pkg::*;

class driver;

    virtual axi4_if.master axi_if;
    mailbox #(signals) gen2drv;
    mailbox #(int)     drv2gen;

    task initialize();

        axi_if.AWADDR = '0;
        axi_if.AWLEN  = '0;
        axi_if.AWSIZE = '0;
        axi_if.AWVALID = 1'b0;
        axi_if.WDATA  = '0;
        axi_if.WLAST  = 1'b0;
        axi_if.WVALID = 1'b0;
        axi_if.BREADY = 1'b0;
        axi_if.ARADDR = '0;
        axi_if.ARLLEN = '0;
        axi_if.ARSIZE = '0;
        axi_if.ARVALID = 1'b0;
        axi_if.RREADY = 1'b0;
        while (axi_if.ARESETn != 1'b1)
            @(posedge axi_if.CLK);
    endtask

endclass
```

DRIVER 1 Driver class setup, interface handle, mailbox, and initialization task.

The write path drives address, burst attributes, write data, WLAST, and response handshakes in protocol order.

```
task write(signals sig);

    @(posedge axi_if.CLK);
    @(posedge axi_if.CLK);

    //Write address
    axi_if.AWADDR <= sig.AWADDR;
    axi_if.AWLEN  <= sig.AWLEN;
    axi_if.AWSIZE <= sig.AWSIZE;
    axi_if.AWVALID <= 1'b1;
    do begin
        @(posedge axi_if.CLK);
    end
    while (!(
        (axi_if.AWREADY === 1'b1) &&
        (axi_if.AWVALID === 1'b1)
    ));
    axi_if.AWVALID <= 1'b0;

    // Write data
    for (int i = 0 ; i < sig.AWLEN + 1 ; i++) begin
        axi_if.WDATA <= sig.WDATA[i];
        axi_if.WVALID <= 1'b1;

        if (i == sig.AWLEN)
            axi_if.WLAST <= 1'b1;
        else
            axi_if.WLAST <= 1'b0;

        do begin
            @(posedge axi_if.CLK);
        end
        while (!(
            (axi_if.WREADY === 1'b1) &&
            (axi_if.WVALID === 1'b1)
        ));
    end
    axi_if.WVALID <= 1'b0;
    axi_if.WLAST  <= 1'b0;

    // Write response
    axi_if.BREADY <= 1'b1;
    do begin
        @(posedge axi_if.CLK);
    end
    while (!(
        (axi_if.BVALID === 1'b1) &&
        (axi_if.BREADY === 1'b1)
    ));
    axi_if.BREADY <= 1'b0;
    @(posedge axi_if.CLK);
    @(posedge axi_if.CLK);
endtask
```

DRIVER 2 AXI write transaction driving sequence.

The read path issues the read address, accepts each returned data beat, and controls RREADY until the burst completes.

```
task read(signals sig);

    // Read address
    axi_if.ARADDR <= sig.ARADDR;
    axi_if.ARLLEN <= sig.ARLLEN;
    axi_if.ARSIZE <= sig.ARSIZE;
    axi_if.ARVALID <= 1'b1;
    do begin
        @(posedge axi_if.CLK);
    end
    while (!(
        (axi_if.ARREADY === 1'b1) &&
        (axi_if.ARVALID === 1'b1)
    ));
    axi_if.ARVALID <= 1'b0;

    // Read data
    axi_if.RREADY <= 1'b1;
    for (int i = 0 ; i < sig.ARLLEN + 1; i++) begin
        do begin
            @(posedge axi_if.CLK);
        end
        while (!(
            (axi_if.RVALID === 1'b1) &&
            (axi_if.RREADY === 1'b1)
        ));
        $display(
            "[DRV][READ] beat=%0d data=%p RLAST=%b",
            i,
            axi_if.RDATA,
            axi_if.RLAST);
    end
    axi_if.RREADY <= 1'b0;
endtask

task drive();
    signals sig;
    initialize();
    forever begin
        gen2drv.get(sig);
        write(sig);
        read(sig);
        drv2gen.put(1);
    end
endtask

endclass

endpackage
```

DRIVER 3 AXI read transaction driving sequence.

The scoreboard acts as the reference model: it validates protocol legality, tracks expected memory contents, compares readback data, and reports verification results.

```
package scb_pkg;
import signals_pkg::*;

class scoreboard;

    mailbox #(signals) mon2scb;

    logic [31:0] reference_memory [0:1023];

    int unsigned writes_checked;
    int unsigned reads_checked;
    int unsigned passed;
    int unsigned failed;
    int unsigned total_transactions;

    function new();
        for (int i = 0; i < 1024; i++)
            reference_memory[i] = 32'd0;

        writes_checked      = 0;
        reads_checked       = 0;
        passed              = 0;
        failed              = 0;
        total_transactions = 0;
    endfunction

    function bit legal_write(signals tr);

        int unsigned beats;
        int unsigned bytes_per_beat;
        int unsigned last_byte_addr;

        beats          = (tr.AWLEN) + 1;
        bytes_per_beat = 1 << tr.AWSIZE;

        last_byte_addr =
            int'(tr.AWADDR) +
            (beats * bytes_per_beat) - 1;

        return (tr.AWSIZE == 3'd2) && (tr.AWADDR[1:0] == 2'b00) && (tr.AWADDR <= 16'h0FFC) &&
            (last_byte_addr <= 16'h0FFF) && ((last_byte_addr >> 2) < 1024);
    endfunction

endclass
```

SCOREBOARD 1 Reference memory, counters, legality checks, and transaction routing.

Scoreboard implementation continued. Each stage contributes to protocol checking, memory modeling, comparison, and final test statistics.

```
function bit legal_read(signals tr);

    int unsigned beats;
    int unsigned bytes_per_beat;
    int unsigned last_byte_addr;

    beats          = (tr.ARLen) + 1;
    bytes_per_beat = 1 << tr.ARSize;

    last_byte_addr =
        int'(tr.ARADDR) +
        (beats * bytes_per_beat) - 1;

    return (tr.ARSize == 3'd2) && (tr.ARADDR[1:0] == 2'b00) && (tr.ARADDR <= 16'h0FFC) &&
        (last_byte_addr <= 16'h0FFF) && ((last_byte_addr >> 2) < 1024);

endfunction

task check_write(signals tr);

    bit transaction_pass;
    bit legal;
    bit data_count_ok;
    logic [1:0] expected_response;
    int unsigned index;
    int unsigned beats;
    writes_checked++;
    transaction_pass = 1'b1;
    beats = (tr.AWLen) + 1;
    legal = legal_write(tr);
    expected_response =
        legal ? 2'b00 : 2'b10;
    index = tr.AWADDR >> 2;

    if (tr.BRESP != expected_response) begin
        $error("[SCB][WRITE] Response mismatch: addr=%h expected=%b actual=%b",
            tr.AWADDR,
            expected_response,
            tr.BRESP);
        transaction_pass = 1'b0;
    end

end
```

SCOREBOARD 2 Write-side checking, burst geometry, and legal-address calculations.

Scoreboard implementation continued. Each stage contributes to protocol checking, memory modeling, comparison, and final test statistics.

```

data_count_ok = (tr.WDATA.size() == beats);

if (!data_count_ok) begin
    $error("[SCB][WRITE] Beat count mismatch: expected=%0d actual=%0d",
           beats,
           tr.WDATA.size());
    transaction_pass = 1'b0;
end

if (legal && (tr.WLAST !== 1'b1)) begin
    $error("[SCB][WRITE] WLAST not asserted on final beat: addr=%h len=%0d",
           tr.AWADDR,
           tr.AWLEN);
    transaction_pass = 1'b0;
end

if (legal && (tr.BRESP === 2'b00) && data_count_ok)
begin
    for (int i = 0; i < beats; i++) begin
        reference_memory[index + i] =
            tr.WDATA[i];
    end
end

if (transaction_pass) begin
    passed++;
    $display("[SCB][WRITE][PASS] addr=%h len=%0d index=%0d data=%p resp=%b",
             tr.AWADDR,
             tr.AWLEN,
             index,
             tr.WDATA,
             tr.BRESP);
end
else begin
    failed++;
end

endtask

```

SCOREBOARD 3 Write-data verification and reference-memory update logic.

Scoreboard implementation continued. Each stage contributes to protocol checking, memory modeling, comparison, and final test statistics.

```
task check_read(signals tr);

    bit transaction_pass;
    bit legal;
    bit data_count_ok;
    logic [1:0] expected_response;
    logic [31:0] expected_data;
    int unsigned index;
    int unsigned beats;
    int unsigned beat_addr;

    reads_checked++;
    transaction_pass = 1'b1;
    beats = int'(tr.ARLEN) + 1;
    legal = legal_read(tr);
    expected_response = legal ? 2'b00 : 2'b10;
    index = tr.ARADDR >> 2;

    if (tr.RRESP != expected_response) begin
        $error("[SCB][READ] Response mismatch: addr=%h expected=%b actual=%b",
            tr.ARADDR,
            expected_response,
            tr.RRESP);
        transaction_pass = 1'b0;
    end

    if (legal) begin
        data_count_ok = (tr.RDATA.size() == beats);
        if (!data_count_ok) begin
            $error("[SCB][READ] Beat count mismatch: expected=%0d actual=%0d",
                beats,
                tr.RDATA.size());
            transaction_pass = 1'b0;
        end
        else begin
            for (int i = 0; i < beats; i++) begin
                expected_data = reference_memory[index + i];
                beat_addr = (tr.ARADDR) + (i << tr.ARSIZE);
                if (tr.RDATA[i] != expected_data) begin
                    $error("[SCB][READ] Data mismatch: beat=%0d addr=%h expected=%p actual=%p",
                        i,
                        beat_addr,
                        expected_data,
                        tr.RDATA[i]);
                    transaction_pass = 1'b0;
                end
            end
        end
    end
end
```

SCOREBOARD 4 Read-side checking against expected reference-memory contents.

Scoreboard implementation continued. Each stage contributes to protocol checking, memory modeling, comparison, and final test statistics.

```

    end else begin
        for (int i = 0; i < tr.RDATA.size(); i++) begin
            if (tr.RDATA[i] !== 32'd0) begin
                $error("[SCB][READ] Invalid read must return zero data: beat=%0d actual=%h",
                    i,
                    tr.RDATA[i]);
                transaction_pass = 1'b0;
            end
        end
    end

    if (tr.RLAST !== 1'b1) begin
        $error("[SCB][READ] RLAST not asserted on final returned beat: addr=%h len=%0d",
            tr.ARADDR,
            tr.ARLen);
        transaction_pass = 1'b0;
    end

    if (transaction_pass) begin
        passed++;
        $display("[SCB][READ][PASS] addr=%h len=%0d index=%0d data=%p resp=%b",
            tr.ARADDR,
            tr.ARLen,
            index,
            tr.RDATA,
            tr.RRESP);
    end
    else begin
        failed++;
    end
endtask

```

SCOREBOARD 5 Result reporting and pass/fail accounting.

Scoreboard implementation continued. Each stage contributes to protocol checking, memory modeling, comparison, and final test statistics.

```

task scoreboard();
    signals actual;
    forever begin
        mon2scb.get(actual);
        case (actual.operation)
            WRITE_OP: check_write(actual);
            READ_OP : check_read(actual);
            default : begin
                failed++;
                $error("[SCB] Unknown transaction operation");
            end
        endcase

        total_transactions++;
    end
endtask

task report();
    $display("=====");
    $display("AXI SCOREBOARD REPORT");
    $display("Writes checked      : %0d", writes_checked);
    $display("Reads checked       : %0d", reads_checked);
    $display("Passed transactions: %0d", passed);
    $display("Failed transactions: %0d", failed);
    $display("=====");
endtask

endclass

endpackage

```

SCOREBOARD 6 Scoreboard run loop and end-of-test summary reporting.

The monitor passively observes AXI handshakes, reconstructs completed transactions, and forwards captured activity to the scoreboard without driving the DUT.

```

4      class monitor;
5
6          virtual axi4_if.monitor axi_if;
7          mailbox #(signals) mon2scb;
8
9          task monitor_write();
10             signals tr;
11             while (axi_if.ARESETn !== 1'b1)
12                 @(posedge axi_if.CLK);
13             forever begin
14                 tr = new();
15                 tr.operation = WRITE_OP;
16                 do begin
17                     @(posedge axi_if.CLK);
18                 end while (!(
19                     (axi_if.ARESETn === 1'b1) &&
20                     (axi_if.AWVALID === 1'b1) &&
21                     (axi_if.AWREADY === 1'b1)
22                 ));
23                 tr.AWADDR = axi_if.AWADDR;
24                 tr.AWLEN  = axi_if.AWLEN;
25                 tr.AWSIZE = axi_if.AWSIZE;
26                 tr.WDATA = new[tr.AWLEN + 1];
27                 for (int i = 0 ; i < tr.AWLEN + 1; i++) begin
28                     do begin
29                         @(posedge axi_if.CLK);
30                     end while (!(
31                         (axi_if.ARESETn === 1'b1) &&
32                         (axi_if.WVALID === 1'b1) &&
33                         (axi_if.WREADY === 1'b1)
34                     ));
35                     tr.WDATA[i] = axi_if.WDATA;
36                     if (i == tr.AWLEN)
37                         tr.WLAST = axi_if.WLAST;
38                 end
39
40                 foreach (tr.WDATA[i]) begin
41                     tr.cg.sample(tr.WDATA[i]);
42                 end
43
44                 tr.WLAST = axi_if.WLAST;
45                 do begin
46                     @(posedge axi_if.CLK);
47                 end while (!(
48                     (axi_if.ARESETn === 1'b1) &&
49                     (axi_if.BVALID === 1'b1) &&
50                     (axi_if.BREADY === 1'b1)
51                 ));
52
53                 tr.BRESP = axi_if.BRESP;
54                 $display("[MON][WRITE] addr=%h data=%p last=%b resp=%b \n",
55                     tr.AWADDR, tr.WDATA, tr.WLAST, tr.BRESP);
56                 tr.cg.sample();
57                 mon2scb.put(tr);
58             end
59         endtask

```

MONITOR 1 Write-channel monitoring and burst-data capture.

The read monitor captures AR-channel information and collects each R-channel data beat until RLAST completes the burst.

```

task monitor_read();
    signals tr;

    while (axi_if.ARESETn !== 1'b1)
        @(posedge axi_if.CLK);

    forever begin
        tr = new();
        tr.operation = READ_OP;
        do begin
            @(posedge axi_if.CLK);
        end while (!(
            (axi_if.ARESETn === 1'b1) &&
            (axi_if.ARVALID === 1'b1) &&
            (axi_if.ARREADY === 1'b1)
        ));

        tr.ARADDR = axi_if.ARADDR;
        tr.ARLen = axi_if.ARLen;
        tr.ARSIZE = axi_if.ARSIZE;

        tr.RDATA = new[tr.ARLen + 1];
        for (int i = 0 ; i < tr.ARLen + 1; i++) begin
            do begin
                @(posedge axi_if.CLK);
            end while (!(
                (axi_if.ARESETn === 1'b1) &&
                (axi_if.RVALID === 1'b1) &&
                (axi_if.RREADY === 1'b1)
            ));
            tr.RDATA[i] = axi_if.RDATA;
            tr.RRESP = axi_if.RRESP;

            if (i == tr.ARLen)
                tr.RLAST = axi_if.RLAST;
        end
        $display("[MON][READ] addr=%h len=%0d data=%p last=%b resp=%b",
            tr.ARADDR,
            tr.ARLen,
            tr.RDATA,
            tr.RLAST,
            tr.RRESP);
        tr.cg.sample();
        mon2scb.put(tr);
    end
endtask

task monitor();
    fork
        monitor_write();
        monitor_read();
    join
endtask

endclass

```

MONITOR 2 Read-channel monitoring and transaction reconstruction.

The environment instantiates the generator, driver, monitor, and scoreboard, connects communication paths, and starts the complete verification flow.

```
package env_pkg;
import monitor_pkg::*;
import driver::*;
import scb_pkg::*;
import signals_pkg::*;
import gen_pkg::*;

class enviroment;
    generator gen;
    driver drv;
    monitor mon;
    scoreboard scb;
    virtual axi4_if.master axi_if;
    virtual axi4_if.monitor axi_mon;
    mailbox #(signals) gen2drv;
    mailbox #(int) drv2gen;
    mailbox #(signals) mon2scb;
    task run_env();
        gen2drv = new(1);
        drv2gen = new(1);
        mon2scb = new();
        gen = new();
        drv = new();
        mon = new();
        scb = new();
        gen.gen2drv = gen2drv;
        gen.drv2gen = drv2gen;
        drv.gen2drv = gen2drv;
        drv.drv2gen = drv2gen;
        mon.mon2scb = mon2scb;
        scb.mon2scb = mon2scb;
        drv.axi_if = axi_if;
        mon.axi_if = axi_mon;
        fork : components
            gen.gen();
            drv.drive();
            mon.monitor();
            scb.scoreboard();
        join_none
        wait (gen.done == 1'b1);
        wait (scb.total_transactions == (2 * gen.num_sig));
        scb.report();
        $stop;
    endtask
endclass
endpackage
```

ENV Environment construction, component wiring, and run control.

The top-level testbench creates the AXI interface, instantiates the DUT and verification environment, and provides the simulation clock/reset infrastructure.

```
import signals_pkg::*;
import gen_pkg::*;
import driver::*;
import monitor_pkg::*;
import scb_pkg::*;
import env_pkg::*;

module axi_tb ();

    bit clk = 1'b0;

    always #5 clk = ~clk;

    axi4_if intrf (.CLK(clk));

    axi4 dut (
        .ACLK(clk),
        .ARESETn(intrf.ARESETn),
        .AWADDR(intrf.AWADDR),
        .AWLEN(intrf.AWLEN),
        .AWSIZE(intrf.AWSIZE),
        .AWVALID(intrf.AWVALID),
        .AWREADY(intrf.AWREADY),
        .WDATA(intrf.WDATA),
        .WLAST(intrf.WLAST),
        .WVALID(intrf.WVALID),
        .WREADY(intrf.WREADY),
        .BRESP(intrf.BRESP),
        .BVALID(intrf.BVALID),
        .BREADY(intrf.BREADY),
        .ARADDR(intrf.ARADDR),
        .ARLEN(intrf.ARLEN),
        .ARSIZE(intrf.ARSIZE),
        .ARVALID(intrf.ARVALID),
        .ARREADY(intrf.ARREADY),
        .RDATA(intrf.RDATA),
        .RRESP(intrf.RRESP),
        .RLAST(intrf.RLAST),
        .RVALID(intrf.RVALID),
        .RREADY(intrf.RREADY)
    );
endmodule
```

TB 1 Top-level module, interface wiring, DUT connections, and simulation setup.

The remaining testbench code completes environment startup and simulation control for the full AXI4 memory verification run.

```
environment env;

initial begin
    intrf.ARESETn = 1'b0;

    repeat (3)
        @(negedge clk);

    intrf.ARESETn = 1'b1;
    @(negedge clk);

    env = new();
    env.axi_if = intrf.master;
    env.axi_mon = intrf.monitor;
    env.run_env();
end

endmodule
```

TB 2 Environment creation, reset sequence, and run invocation.

PROJECT SUMMARY

AXI4 memory verification flow completed

The final environment separates stimulus generation, protocol driving, passive monitoring, checking, and top-level orchestration into reusable SystemVerilog verification components.

These logs and waveform apply for different test cases, which covered all ranges of beat lengths with different addresses

```
# [DRV] [READ] beat=0 data=2210642852 RLAST=0
# [DRV] [READ] beat=1 data=2739255796 RLAST=0
# [DRV] [READ] beat=2 data=829655252 RLAST=0
# [DRV] [READ] beat=3 data=2730258368 RLAST=0
# [DRV] [READ] beat=4 data=3289234374 RLAST=0
# [DRV] [READ] beat=5 data=3750851088 RLAST=0
# [DRV] [READ] beat=6 data=1336245774 RLAST=0
# [DRV] [READ] beat=7 data=2075917209 RLAST=0
# [DRV] [READ] beat=8 data=3578526410 RLAST=0
# [DRV] [READ] beat=9 data=1148476141 RLAST=0
# [DRV] [READ] beat=10 data=3292182357 RLAST=0
# [DRV] [READ] beat=11 data=3917049909 RLAST=0
# [DRV] [READ] beat=12 data=3055763768 RLAST=0
# [DRV] [READ] beat=13 data=1162829302 RLAST=0
# [DRV] [READ] beat=14 data=2169253640 RLAST=0
# [DRV] [READ] beat=15 data=2295887306 RLAST=0
# [DRV] [READ] beat=16 data=3010726603 RLAST=0
# [DRV] [READ] beat=17 data=2352552594 RLAST=0
# [DRV] [READ] beat=18 data=1675822561 RLAST=0
# [DRV] [READ] beat=19 data=4238528214 RLAST=0
# [DRV] [READ] beat=20 data=2892739181 RLAST=0
# [DRV] [READ] beat=21 data=1222552124 RLAST=0
# [DRV] [READ] beat=22 data=1325724445 RLAST=0
# [DRV] [READ] beat=23 data=3867437876 RLAST=0
# [DRV] [READ] beat=24 data=538460969 RLAST=0
# [DRV] [READ] beat=25 data=1903405921 RLAST=0
# [DRV] [READ] beat=26 data=3022376563 RLAST=0
# [MON] [READ] addr=0a14 len=27 data='{2210642852, 2739255796, 829655252, 2730258368, 3289234374, 3750851088, 1336245774, 2075917209, 3578526410, 1148476141, 3292182357, 3917049909, 3055763768, 1162829302, 2169253640, 2295887306, 3010726603, 2352552594, 1675822561, 4238528214, 2892739181, 1222552124, 1325724445, 3867437876, 538460969, 1903405921, 3022376563, 813253004} last=1 resp=0
0
# [DRV] [READ] beat=27 data=813253004 RLAST=1
# [SCB] [READ] [PASS] addr=0a14 len=27 index=645 data='{2210642852, 2739255796, 829655252, 2730258368, 3289234374, 3750851088, 1336245774, 2075917209, 3578526410, 1148476141, 3292182357, 3917049909, 3055763768, 1162829302, 2169253640, 2295887306, 3010726603, 2352552594, 1675822561, 4238528214, 2892739181, 1222552124, 1325724445, 3867437876, 538460969, 1903405921, 3022376563, 813253004} resp=00
```

LOG 1

Read burst simulation log

```
# [GEN] write_addr=09e4 write_data='{2833739227, 2222577614, 1113790909, 3491637220, 2619676938, 4287224247, 413277671, 2427169368, 818197848, 2112377381, 948232269, 4201650191, 347949722, 674531719, 1932968159, 3489254850, 2818298951, 3790667513, 2639422104, 3018857457, 1082413264, 2304535647, 1670974407, 3256132495, 2841538678, 1375065161, 2911806056, 3293903204, 367246342, 2567033438, 3302683344, 1771369674, 87790385, 944624780, 4028020137, 4283009321, 2763379796, 1711274330, 3340242382, 3338600456, 3067696352, 4286394848, 3389842674, 3550819987, 779598204, 547404467, 341524076, 1671610477, 2531461321, 3003465824, 3729093895, 2733867159, 1356621860, 4028710621, 3312760144, 1740313397, 3000305688, 2243541253, 3725889862, 3809485977, 3181983025, 4109769510, 203827818, 2784151931, 738608964, 1189325080, 539704709, 3738776861, 2998889301, 2683724167, 1966063033, 3632457119, 1047568991, 2902774293, 2013639884, 2519676422, 4212261488, 3346895170, 2260379124, 1462757052, 1378086627, 1057891568, 3432822043, 1264040718, 4228515202, 2651498549, 4072364797, 2177530478, 220350849, 2976644169, 3428441528, 1348308747, 107688053, 2580649453, 1886078152, 3820187604, 2673466705, 2832329689, 3607959864, 1226229666, 922407315, 4136283645, 1694820210} read_addr=0004
# [MON] [WRITE] addr=09e4 data='{2833739227, 2222577614, 1113790909, 3491637220, 2619676938, 4287224247, 413277671, 2427169368, 818197848, 2112377381, 948232269, 4201650191, 347949722, 674531719, 1932968159, 3489254850, 2818298951, 3790667513, 2639422104, 3018857457, 1082413264, 2304535647, 1670974407, 3256132495, 2841538678, 1375065161, 2911806056, 3293903204, 367246342, 2567033438, 3302683344, 1771369674, 87790385, 944624780, 4028020137, 4283009321, 2763379796, 1711274330, 3340242382, 3338600456, 3067696352, 4286394848, 3389842674, 3550819987, 779598204, 547404467, 341524076, 1671610477, 2531461321, 3003465824, 3729093895, 2733867159, 1356621860, 4028710621, 3312760144, 1740313397, 3000305688, 2243541253, 3725889862, 3809485977, 3181983025, 4109769510, 203827818, 2784151931, 738608964, 1189325080, 539704709, 3738776861, 2998889301, 2683724167, 1966063033, 3632457119, 1047568991, 2902774293, 2013639884, 2519676422, 4212261488, 3346895170, 2260379124, 1462757052, 1378086627, 1057891568, 3432822043, 1264040718, 4228515202, 2651498549, 4072364797, 2177530478, 220350849, 2976644169, 3428441528, 1348308747, 107688053, 2580649453, 1886078152, 3820187604, 2673466705, 2832329689, 3607959864, 1226229666, 922407315, 4136283645, 1694820210} last=1 resp=00
#
# [SCB] [WRITE] [PASS] addr=09e4 len=102 index=633 data='{2833739227, 2222577614, 1113790909, 3491637220, 2619676938, 4287224247, 413277671, 2427169368, 818197848, 2112377381, 948232269, 4201650191, 347949722, 674531719, 1932968159, 3489254850, 2818298951, 3790667513, 2639422104, 3018857457, 1082413264, 2304535647, 1670974407, 3256132495, 2841538678, 1375065161, 2911806056, 3293903204, 367246342, 2567033438, 3302683344, 1771369674, 87790385, 944624780, 4028020137, 4283009321, 2763379796, 1711274330, 3340242382, 3338600456, 3067696352, 4286394848, 3389842674, 3550819987, 779598204, 547404467, 341524076, 1671610477, 2531461321, 3003465824, 3729093895, 2733867159, 1356621860, 4028710621, 3312760144, 1740313397, 3000305688, 2243541253, 3725889862, 3809485977, 3181983025, 4109769510, 203827818, 2784151931, 738608964, 1189325080, 539704709, 3738776861, 2998889301, 2683724167, 1966063033, 3632457119, 1047568991, 2902774293, 2013639884, 2519676422, 4212261488, 3346895170, 2260379124, 1462757052, 1378086627, 1057891568, 3432822043, 1264040718, 4228515202, 2651498549, 4072364797, 2177530478, 220350849, 2976644169, 3428441528, 1348308747, 107688053, 2580649453, 1886078152, 3820187604, 2673466705, 2832329689, 3607959864, 1226229666, 922407315, 4136283645, 1694820210} resp=00
```

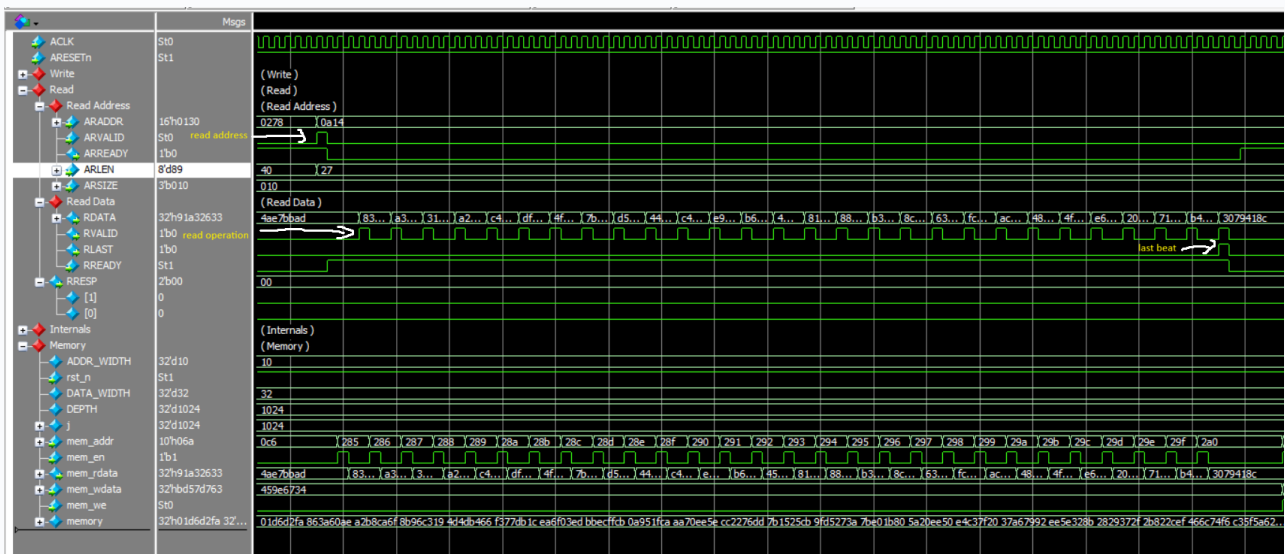
LOG 2

Write burst simulation log

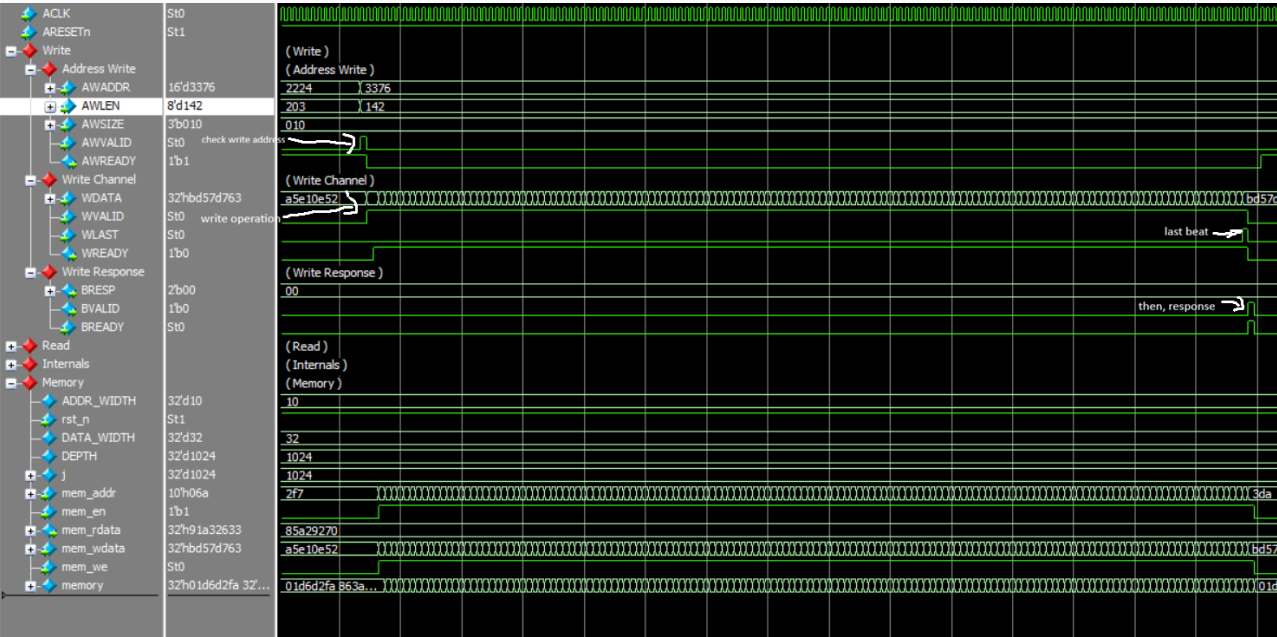
```
# =====
# AXI SCOREBOARD REPORT
# Writes checked      : 3000
# Reads checked       : 3000
# Passed transactions: 6000
# Failed transactions: 0
# =====
```

LOG 3

AXI scoreboard report



READ WAVEFORM Read transaction waveform



WRITE WAVEFORM Write transaction waveform

1. Functional Coverage

/signals_pkg/signals	100.00%				
TYPE cg	100.00%	100	100.00...		✓
CVP cg::c_AWADDR	100.00%	100	100.00...		✓
bin b1	50955	1	100.00...		✓
bin b2	45789	1	100.00...		✓
bin b3	48058	1	100.00...		✓
bin b4	46790	1	100.00...		✓
bin b5	47981	1	100.00...		✓
bin b6	49400	1	100.00...		✓
bin b7	54299	1	100.00...		✓
bin b8	48233	1	100.00...		✓
CVP cg::c_ARADDR	100.00%	100	100.00...		✓
bin b1	358	1	100.00...		✓
bin b2	379	1	100.00...		✓
bin b3	369	1	100.00...		✓
bin b4	384	1	100.00...		✓
bin b5	378	1	100.00...		✓
bin b6	360	1	100.00...		✓
bin b7	408	1	100.00...		✓
bin b8	364	1	100.00...		✓
CVP cg::c_AWLEN	100.00%	100	100.00...		✓
bin b0	26	1	100.00...		✓
bin b1	23756	1	100.00...		✓
bin b2	73338	1	100.00...		✓
bin b3	118839	1	100.00...		✓
bin b4	175546	1	100.00...		✓
CVP cg::c_ARLEN	100.00%	100	100.00...		✓
bin b0	16	1	100.00...		✓
bin b1	746	1	100.00...		✓
bin b2	714	1	100.00...		✓
bin b3	793	1	100.00...		✓
bin b4	731	1	100.00...		✓
CVP cg::c_WDATA	100.00%	100	100.00...		✓
bin b1	54600	1	100.00...		✓
bin b2	48475	1	100.00...		✓
bin b3	48433	1	100.00...		✓
bin b4	48505	1	100.00...		✓
bin b5	49094	1	100.00...		✓
bin b6	48266	1	100.00...		✓
bin b7	48570	1	100.00...		✓
bin b8	48562	1	100.00...		✓

FUNCTIONAL COVERAGE Coverage view 1

1. Functional Coverage - Detailed Views

CVP cgrsic_RESP	100.00%	100	100.00...	✓
bin b1	2611	1	100.00...	✓
bin b2	389	1	100.00...	✓
CVP cgrsic_RESP	100.00%	100	100.00...	✓
bin b1	2642	1	100.00...	✓
bin b2	358	1	100.00...	✓
CVP cgrsic_AWSIZE	100.00%	100	100.00...	✓
bin b1	391505	1	100.00...	✓
CVP cgrsic_ARSIZE	100.00%	100	100.00...	✓
bin b1	3000	1	100.00...	✓
CROSS cgrsroos_wire_add_len	100.00%	100	100.00...	✓
bin <b8,b4>	18961	1	100.00...	✓
bin <b7,b4>	25497	1	100.00...	✓
bin <b6,b4>	22451	1	100.00...	✓
bin <b5,b4>	23071	1	100.00...	✓
bin <b4,b4>	20380	1	100.00...	✓
bin <b3,b4>	21714	1	100.00...	✓
bin <b2,b4>	20365	1	100.00...	✓
bin <b1,b4>	23107	1	100.00...	✓
bin <b8,b3>	16907	1	100.00...	✓
bin <b7,b3>	16247	1	100.00...	✓
bin <b6,b3>	15620	1	100.00...	✓
bin <b5,b3>	13051	1	100.00...	✓
bin <b4,b3>	14219	1	100.00...	✓
bin <b3,b3>	14946	1	100.00...	✓
bin <b2,b3>	12854	1	100.00...	✓
bin <b1,b3>	14995	1	100.00...	✓
bin <b8,b2>	9440	1	100.00...	✓
bin <b7,b2>	9685	1	100.00...	✓
bin <b6,b2>	8899	1	100.00...	✓
bin <b5,b2>	8677	1	100.00...	✓
bin <b4,b2>	9473	1	100.00...	✓
bin <b3,b2>	8686	1	100.00...	✓
bin <b2,b2>	8869	1	100.00...	✓
bin <b1,b2>	9609	1	100.00...	✓
bin <b8,b1>	2919	1	100.00...	✓
bin <b7,b1>	2868	1	100.00...	✓
bin <b6,b1>	2428	1	100.00...	✓
bin <b5,b1>	3180	1	100.00...	✓
bin <b4,b1>	2716	1	100.00...	✓
bin <b3,b1>	2708	1	100.00...	✓
bin <b2,b1>	3697	1	100.00...	✓
bin <b1,b1>	3240	1	100.00...	✓
bin <b8,b0>	6	1	100.00...	✓

FUNCTIONAL COVERAGE Coverage view 2

CROSS cgrsroos_read_add_len	100.00%	100	100.00...	✓
bin <b8,b4>	79	1	100.00...	✓
bin <b7,b4>	86	1	100.00...	✓
bin <b6,b4>	94	1	100.00...	✓
bin <b5,b4>	91	1	100.00...	✓
bin <b4,b4>	96	1	100.00...	✓
bin <b3,b4>	105	1	100.00...	✓
bin <b2,b4>	85	1	100.00...	✓
bin <b1,b4>	95	1	100.00...	✓
bin <b8,b3>	97	1	100.00...	✓
bin <b7,b3>	119	1	100.00...	✓
bin <b6,b3>	94	1	100.00...	✓
bin <b5,b3>	92	1	100.00...	✓
bin <b4,b3>	90	1	100.00...	✓
bin <b3,b3>	95	1	100.00...	✓
bin <b2,b3>	102	1	100.00...	✓
bin <b1,b3>	104	1	100.00...	✓
bin <b8,b2>	87	1	100.00...	✓
bin <b7,b2>	100	1	100.00...	✓
bin <b6,b2>	77	1	100.00...	✓
bin <b5,b2>	96	1	100.00...	✓
bin <b4,b2>	96	1	100.00...	✓
bin <b3,b2>	89	1	100.00...	✓
bin <b2,b2>	92	1	100.00...	✓
bin <b1,b2>	77	1	100.00...	✓
bin <b8,b1>	99	1	100.00...	✓
bin <b7,b1>	101	1	100.00...	✓
bin <b6,b1>	94	1	100.00...	✓
bin <b5,b1>	97	1	100.00...	✓
bin <b4,b1>	99	1	100.00...	✓
bin <b3,b1>	78	1	100.00...	✓
bin <b2,b1>	99	1	100.00...	✓
bin <b1,b1>	79	1	100.00...	✓
bin <b8,b0>	2	1	100.00...	✓
bin <b7,b0>	2	1	100.00...	✓
bin <b6,b0>	1	1	100.00...	✓
bin <b5,b0>	2	1	100.00...	✓
bin <b4,b0>	3	1	100.00...	✓
bin <b3,b0>	2	1	100.00...	✓
bin <b2,b0>	1	1	100.00...	✓
bin <b1,b0>	3	1	100.00...	✓

FUNCTIONAL COVERAGE Coverage view 3

2. Memory Code Coverage

```
=== Instance: /axi_tb/dut/mem_inst
=== Design Unit: work.axi_memory
=====
Branch Coverage:
  Enabled Coverage      Bins    Hits    Misses  Coverage
  -----
  Branches              5      5      0    100.00%

=====Branch Details=====
Branch Coverage for instance /axi_tb/dut/mem_inst

  Line    Item              Count    Source
  ----    -
  File axi_memory.v
  -----IF Branch-----
  24              1      978296    Count coming in to IF
  24              1        2
  26              1     649051
  26              1     329243    All False Count
Branch totals: 3 hits of 3 branches = 100.00%

  -----IF Branch-----
  27              1     649051    Count coming in to IF
  27              1     322420
  29              1     326631
Branch totals: 2 hits of 2 branches = 100.00%
```

BRANCH COVERAGE Memory module

```
Statement Coverage:
  Enabled Coverage      Bins    Hits    Misses  Coverage
  -----
  Statements            7      7      0    100.00%

=====Statement Details=====
Statement Coverage for instance /axi_tb/dut/mem_inst --

  Line    Item              Count    Source
  ----    -
  File axi_memory.v
  23              1      978296
  25              1        2
  28              1     322420
  30              1     326631
  36              1        1
  36              2      1024
  37              1      1024
```

STATEMENT COVERAGE Memory module

```
Toggle Coverage:
  Enabled Coverage      Bins    Hits    Misses  Coverage
  -----
  Toggles              154    154      0    100.00%

=====Toggle Details=====
Toggle Coverage for instance /axi_tb/dut/mem_inst --

  Node    1H->0L    0L->1H    "Coverage"
  -----
  Total Node Count = 77
  Toggled Node Count = 77
  Untoggled Node Count = 0
  Toggle Coverage = 100.00% (154 of 154 bins)
```

TOGGLE COVERAGE Memory module

3. DUT Coverage - Statement Coverage

```

Statement Coverage:
Enabled Coverage      Bins   Hits   Misses Coverage
-----
Statements            85     85     0    100.00%

=====Statement Details=====

Statement Coverage for instance /axi_tb/dut --

Line   Item      Count   Source
----   -
File axi4.v
60      1          3
61      1          3
64      1    388507
65      1    385247
69      1    388503
70      1    385245
103     1    1571243
106     1          3
107     1          3
108     1          3
109     1          3
111     1          3
112     1          3
113     1          3
114     1          3
115     1          3
118     1          3
119     1          3
120     1          3
121     1          3
122     1          3
123     1          3
126     1          3
127     1          3
128     1          3
129     1          3
130     1          3
131     1          3
132     1          3

```

STATEMENT COVERAGE DUT view 1

```

133     1          3
135     1          3
139     1    1571240
140     1    1571240
147     1    1176735
148     1    1176735
149     1    1176735
153     1    3000
154     1    3000
155     1    3000
156     1    3000
158     1    3000
159     1    3000
165     1    3000
166     1    3000
174     1    322420
175     1    322420
176     1    322420
177     1    322420
182     1    3000
183     1    3000
187     1    389
189     1    2611
191     1    3000
194     1    385505
195     1    385505
202     1    3000
203     1    3000
204     1    3000
216     1    412505
217     1    412505
218     1    412505
222     1    3000
223     1    3000
224     1    3000
225     1    3000
227     1    3000
228     1    3000
235     1    2642
236     1    2642
238     1    3000

```

STATEMENT COVERAGE DUT view 2

3. DUT Coverage - Statement & Branch

238	1	3000
241	1	385245
246	1	653262
247	1	653262
249	1	117228
250	1	117228
253	1	770490
254	1	770490
257	1	385245
261	1	382245
262	1	382245
266	1	323989
267	1	323989
269	1	382245
273	1	3000
274	1	3000

STATEMENT COVERAGE DUT view 3

```
=====
=== Instance: /axi_tb/dut
=== Design Unit: work.axi4
=====
Branch Coverage:
  Enabled Coverage      Bins    Hits    Misses  Coverage
  -----
  Branches              30      30      0    100.00%

=====Branch Details=====
Branch Coverage for instance /axi_tb/dut

  Line    Item              Count    Source
  ---    --
  File axi4.v
  -----IF Branch-----
    104              1571243    Count coming in to IF
    104          1              3
    137          1              1571240
  Branch totals: 2 hits of 2 branches = 100.00%

  -----CASE Branch-----
    145              1571240    Count coming in to CASE
    146          1              1176735
    163          1              3000
    169          1              388505
    200          1              3000
  Branch totals: 4 hits of 4 branches = 100.00%

  -----IF Branch-----
    151              1176735    Count coming in to IF
    151          1              3000
    151              1173735    All False Count
  Branch totals: 2 hits of 2 branches = 100.00%

  -----IF Branch-----
    172              388505    Count coming in to IF
    172          1              322420
    172              66085    All False Count
  Branch totals: 2 hits of 2 branches = 100.00%
```

BRANCH COVERAGE DUT view 1

3. DUT Coverage - Branch Coverage

```

-----IF Branch-----
181          388505      Count coming in to IF
181          1          3000
192          1          385505
Branch totals: 2 hits of 2 branches = 100.00%

-----IF Branch-----
186          3000      Count coming in to IF
186          1          389
188          1          2611
Branch totals: 2 hits of 2 branches = 100.00%

-----CASE Branch-----
214          1571240    Count coming in to CASE
215          1          412505
232          1          3000
240          1          385245
243          1          770490
Branch totals: 4 hits of 4 branches = 100.00%

-----IF Branch-----
220          412505    Count coming in to IF
220          1          3000
                409505    All False Count
Branch totals: 2 hits of 2 branches = 100.00%

-----IF Branch-----
234          3000      Count coming in to IF
234          1          2642
                358      All False Count
Branch totals: 2 hits of 2 branches = 100.00%

-----IF Branch-----
245          770490    Count coming in to IF
245          1          653262
248          1          117228
Branch totals: 2 hits of 2 branches = 100.00%

```

BRANCH COVERAGE DUT view 2

```

-----IF Branch-----
256          770490    Count coming in to IF
256          1          385245
                385245    All False Count
Branch totals: 2 hits of 2 branches = 100.00%

-----IF Branch-----
259          385245    Count coming in to IF
259          1          382245
271          1          3000
Branch totals: 2 hits of 2 branches = 100.00%

-----IF Branch-----
265          382245    Count coming in to IF
265          1          323989
                58256    All False Count
Branch totals: 2 hits of 2 branches = 100.00%

```

BRANCH COVERAGE DUT view 3

3. DUT Coverage - Condition & Expression

```

=====Condition Details=====

Condition Coverage for instance /axi_tb/dut --

File axi4.v
-----Focused Condition View-----
Line      172 Item      1  (write_addr_valid && ~write_boundary_cross)
Condition totals: 2 of 2 input terms covered = 100.00%

-----Focused Condition View-----
Line      245 Item      1  (read_addr_valid && ~read_boundary_cross)
Condition totals: 2 of 2 input terms covered = 100.00%

-----Focused Condition View-----
Line      259 Item      1  (read_burst_cnt > 0)
Condition totals: 1 of 1 input term covered = 100.00%

-----Focused Condition View-----
Line      265 Item      1  (read_addr_valid && ~read_boundary_cross)
Condition totals: 2 of 2 input terms covered = 100.00%

```

CONDITION COVERAGE DUT

```

Expression Coverage:
Enabled Coverage
-----
Expressions          5          5          0  100.00%

=====Expression Details=====

Expression Coverage for instance /axi_tb/dut --

File axi4.v
-----Focused Expression View-----
Line      64 Item      1  (((write_addr & 4095) + (write_burst_cnt << write_size)) > 4095)
Expression totals: 1 of 1 input term covered = 100.00%

-----Focused Expression View-----
Line      65 Item      1  (((read_addr & 4095) + (read_burst_cnt << read_size)) > 4095)
Expression totals: 1 of 1 input term covered = 100.00%

-----Focused Expression View-----
Line      69 Item      1  ((write_addr >> 2) < 1024)
Expression totals: 1 of 1 input term covered = 100.00%

-----Focused Expression View-----
Line      70 Item      1  ((read_addr >> 2) < 1024)
Expression totals: 1 of 1 input term covered = 100.00%

-----Focused Expression View-----
Line     254 Item      1  (read_burst_cnt == 0)
Expression totals: 1 of 1 input term covered = 100.00%

```

EXPRESSION COVERAGE DUT

3. DUT Coverage - FSM Coverage

```
=====FSM Details=====
FSM Coverage for instance /axi_tb/dut --
FSM_ID: write_state
Current State Object : write_state
-----
State Value MapInfo :
-----
Line      State Name      Value
-----
146        W_IDLE         0
163        W_ADDR         1
169        W_DATA         2
200        W_RESP         3
Covered States :
-----
State      Hit_count
-----
W_IDLE     3003
W_ADDR     3000
W_DATA     3000
W_RESP     3000
Covered Transitions :
-----
Line      Trans_ID      Hit_count      Transition
-----
159        0              3000           W_IDLE -> W_ADDR
166        1              3000           W_ADDR -> W_DATA
183        3              3000           W_DATA -> W_RESP
204        5              3000           W_RESP -> W_IDLE

Summary
-----
FSM States      Bins    Hits    Misses  Coverage
-----
FSM States      4        4        0    100.00%
FSM Transitions 4        4        0    100.00%
FSM_ID: read_state
```

FSM COVERAGE DUT view 1

```
Current State Object : read_state
-----
State Value MapInfo :
-----
Line      State Name      Value
-----
215        R_IDLE         0
232        R_ADDR         1
240        R_WAIT         2
243        R_DATA         3
Covered States :
-----
State      Hit_count
-----
R_IDLE     3003
R_ADDR     3000
R_WAIT     385245
R_DATA     385245
Covered Transitions :
-----
Line      Trans_ID      Hit_count      Transition
-----
228        0              3000           R_IDLE -> R_ADDR
238        1              3000           R_ADDR -> R_WAIT
241        3              385245          R_WAIT -> R_DATA
274        5              3000           R_DATA -> R_IDLE
269        6              382245          R_DATA -> R_WAIT

Summary
-----
FSM States      Bins    Hits    Misses  Coverage
-----
FSM States      4        4        0    100.00%
FSM Transitions 5        5        0    100.00%
```

FSM COVERAGE DUT view 2

3. DUT Coverage - Toggle Coverage

```
Toggle Coverage:
  Enabled Coverage      Bins    Hits    Misses  Coverage
  -----
  Toggles               474    474         0   100.00%

=====Toggle Details=====

Toggle Coverage for instance /axi_tb/dut --

                                Node      1H->0L      0L->1H  "Coverage"
                                -----

Total Node Count      =      237
Toggled Node Count    =      237
Untoggled Node Count  =         0

Toggle Coverage      =    100.00% (474 of 474 bins)
```

TOGGLE COVERAGE DUT

4. Exclusions and Justification

JUSTIFICATION

The excluded points are structurally unreachable or constant under the legal AXI4 configuration used in this project (for example aligned addresses, fixed transfer size, and invalid FSM transitions). Excluding them prevents impossible cases from reducing the meaningful coverage score.

```
coverage exclude -src axi4.v -code s -line 208 279
coverage exclude -src axi4.v -code b -line 170 201-208 279
coverage exclude -src axi4.v -code c -line 151-170 181-234 256
coverage exclude -du axi4 -ft write_state W_ADDR->W_IDLE W_DATA->W_IDLE
coverage exclude -du axi4 -ft read_state R_ADDR->R_IDLE R_WAIT->R_IDLE
coverage exclude -scope /axi_tb/dut -togglenode {ARADDR[0]} {ARADDR[1]} {ARADDR[12]} {ARADDR[13]}
{ARADDR[14]} {ARADDR[15]} ARESETn {ARSIZE[0]} {ARSIZE[1]} {ARSIZE[2]}
coverage exclude -scope /axi_tb/dut -togglenode {AWADDR[0]} {AWADDR[1]} {AWADDR[12]} {AWADDR[13]}
{AWADDR[14]} {AWADDR[15]} {AWSIZE[0]} {AWSIZE[1]} {AWSIZE[2]} BRESP(0)
coverage exclude -scope /axi_tb/dut -togglenode RRESP(0) RVALID WREADY WVALID mem_rdata_reg read_addr(15) read_addr(14) read_addr(13) read_addr(1) read_addr(0)
coverage exclude -scope /axi_tb/dut -togglenode read_addr_incr[0] read_addr_incr[1] read_addr_incr[2] read_addr_incr[3]
```

EXCLUSIONS Part 1

```
coverage exclude -scope /axi_tb/dut -togglenode RRESP(0) RVALID WREADY WVALID mem_rdata_reg read_addr(15) read_addr(14) read_addr(13) read_addr(1) read_addr(0)
coverage exclude -scope /axi_tb/dut -togglenode {read_addr_incr[0]} {read_addr_incr[1]} {read_addr_incr[2]} {read_addr_incr[3]}
{read_addr_incr[4]} {read_addr_incr[5]} {read_addr_incr[6]} {read_addr_incr[7]} {read_addr_incr[8]} {read_addr_incr[9]}
coverage exclude -scope /axi_tb/dut -togglenode {read_addr_incr[10]} {read_addr_incr[11]} {read_addr_incr[12]} {read_addr_incr[13]}
{read_addr_incr[14]} {read_addr_incr[15]} read_addr_valid read_size(2) read_size(1) read_size(0)
coverage exclude -scope /axi_tb/dut -togglenode read_state(2) write_addr {write_addr_incr[0]} {write_addr_incr[1]}
{write_addr_incr[2]} {write_addr_incr[3]} {write_addr_incr[4]} {write_addr_incr[5]} {write_addr_incr[6]} {write_addr_incr[7]}
coverage exclude -scope /axi_tb/dut -togglenode {write_addr_incr[8]} {write_addr_incr[9]} {write_addr_incr[10]}
{write_addr_incr[11]} {write_addr_incr[12]} {write_addr_incr[13]} {write_addr_incr[14]} {write_addr_incr[15]} write_boundary_cross write_size(2)
coverage exclude -scope /axi_tb/dut -togglenode write_size(1) write_size(0) write_state(2)
coverage exclude -scope /axi_tb/dut/mem_inst -togglenode j
```

EXCLUSIONS Part 2

Coverage Summary

```
TOTAL COVERGROUP COVERAGE: 100.00%  COVERGROUP TYPES: 1
Total Coverage By Instance (filtered view): 100.00%
```

OVERALL COVERAGE

Verification Scripts

Simulation command files used during verification.

```
vlog axi_memory.v memory_tb.sv
vsim -voptargs=+acc work.memory_tb
add wave *
add wave -position insertpoint \
sim:/memory_tb/memory
run -all
```

MEM DO This script was used to test the memory

```
vlog *.v +cover -covercells
vlog interface.sv signal.sv generator.sv driver.sv monitor.sv scoreboard.sv enviroment.sv axi_tb.sv +cover=f -covercells
vsim -voptargs=+acc work.axi_tb -cover
do exclude.do
coverage save -onexit cov.ucdb
do wave.do
run -all
coverage report -details -output cov_report.txt
```

AXI DO This script was used to test the AXI4